SYSTEMS THINKING APPLIED TO A REAL PRODUCT

Samaan Bol GST through a systems lens

Six systems-thinking tools, each drawn from what the Samaan Bol web and mobile apps actually do: stock-and-flow, feedback loops, behaviour over time, archetype traps, leverage points and working guidelines. The aim is to show why the GST feature behaves the way it does and where a change would matter most.

Prepared	10 October 2026
Based on	Web repo `dukaan-voice-legal` at commit c4770c9, mobile repo `dukaan_voice_mobile` (GST audit, scope classification and acceptance register), and the web parity plan of 7–8 October 2026
Framework	Donella Meadows, *Thinking in Systems*. The tools are hers. Every application to Samaan Bol is my own reading, not a quotation.

How to read the evidence tags

CODE Read directly from source (for example the shared tax core in `app/lib/shop/gst-core/gst.ts`).

DOCS Stated in the project's own plans, audits or acceptance register.

INFERENCE My interpretation. It fits the code and docs but has not been measured in real shops. Treat as a hypothesis to test.

The system in one paragraph

A shopkeeper sells goods all day. The GST feature has to turn each sale into a correct, numbered, permanent tax invoice, survive bad networks and shared devices, and leave records that an accountant can later use to file. The scope is deliberately narrow: same-state counter sales, with unsupported cases such as interstate sales and e-invoicing *blocked* rather than guessed. Samaan Bol prepares files for external filing; it does not file returns.

What is inside and outside the boundary

INSIDE (MODELLED HERE)	OUTSIDE (SHOWN AS CLOUDS)
Product tax profiles, invoice number blocks, issued documents, the on-device journal of uncertain requests, record checks, period and export history.	The GST portal and return filing, the merchant's accountant, the source of correct HSN codes and rates, other billing tools, the customers who receive bills.

TOOL 1

Stock-and-flow diagram

A stock is something that accumulates and can be counted at a moment in time. A flow changes it. Clouds mark where the model stops caring. Reading Samaan Bol this way gives five stocks that matter for GST. Everything else (screens, reports) is a view of one of these.

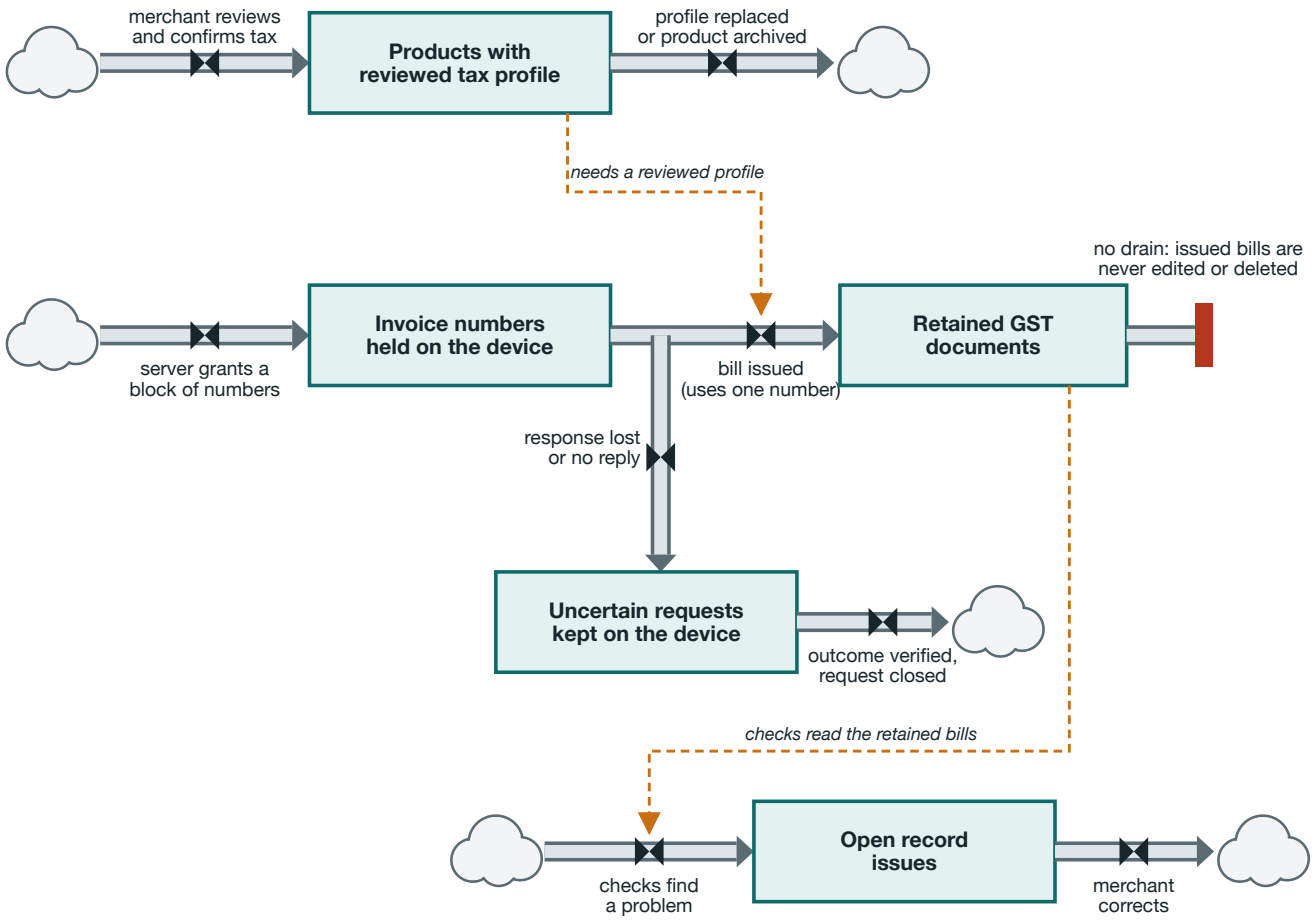


Figure 1. The GST bill pipeline. Boxes are stocks, pipes with valves are flows, clouds are the boundary, dashed arrows carry information (they move no material). The red end-cap on Retained GST documents marks a stock with no outflow by design.

Box a stock you could count today

Pipe with valve a flow; the valve is the rule or action that sets its rate

Cloud source or sink outside the boundary

Dashed arrow information that changes a flow, not material

What each stock and flow means in the project

STOCK	INFLOW	OUTFLOW	WHERE IT LIVES
Products with reviewed tax profile	Merchant confirms category, HSN/SAC code and rate (product form, bulk import)	Profile replaced by a new version, or product archived	`validateProductTax` in `gst.ts` ; `/shop/products/gst` CODE

Invoice numbers held on the device	Server grants a block; a number has a financial-year prefix and an index from 1 to 100	One number used per issued bill	`allocationNumber` in `gst.ts`; `/shop/settings/gst/numbers` CODE
Retained GST documents	Invoices, and credit or debit notes issued as separate documents	None. Corrections add a new document	`gst-document.ts`, `gst-issuance.ts` CODE DOCS
Uncertain requests kept on the device	A financial request whose reply was lost	Outcome verified against the original request, then closed	`gst-recovery.ts`, `gst-storage.ts` (IndexedDB journal); `/shop/settings/gst/recovery` CODE
Open record issues	Checks find incomplete profiles, missing documents, number conflicts	Merchant corrects the record	`/shop/settings/gst/checks` DOCS

What the structure tells you

Retained documents only fill. Because nothing drains them, correctness at the inflow valve is everything: a wrong bill cannot be removed, only offset by a credit note. That is why the tax calculation, the profile gate and the number allocation are so strict.

The profile gate controls the bill valve. The dashed link from products to the "bill issued" valve is the real rule: when a registered shop sells a product with no reviewed profile, the core refuses with `tax_profile_missing`. Readiness of one stock sets the rate of another.

Uncertain requests are a side tank, not a leak. A lost reply does not discard the sale. It moves into a holding stock until the outcome is verified, which is how the web app avoids creating two bills for one sale.

Feedback loop diagram

Loops explain why stocks behave as they do. A **balancing** loop pulls a stock toward a goal. A **reinforcing** loop amplifies change in one direction. The three balancing loops below sit directly on the stock-and-flow picture from Figure 1, one on each stock they regulate.

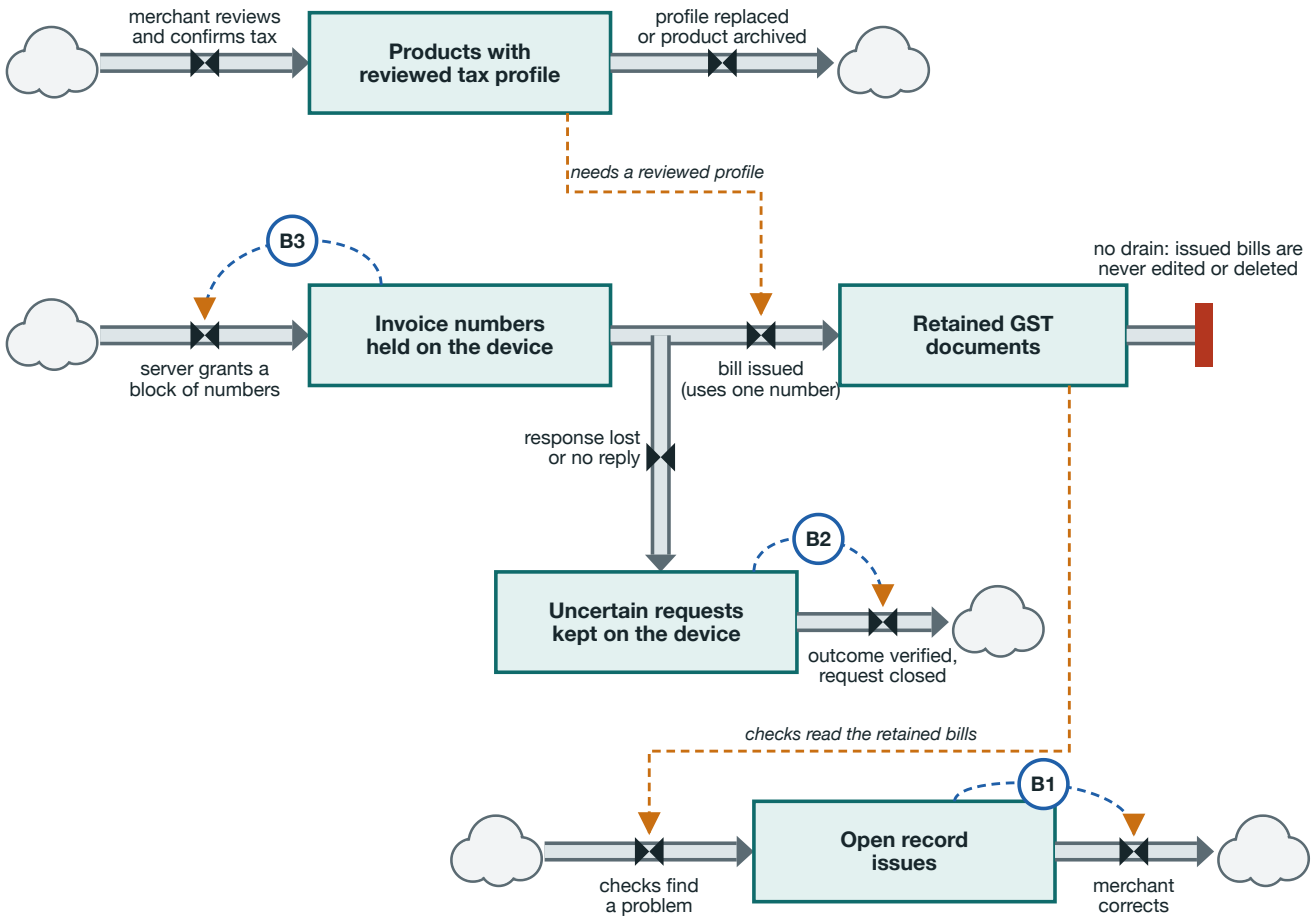
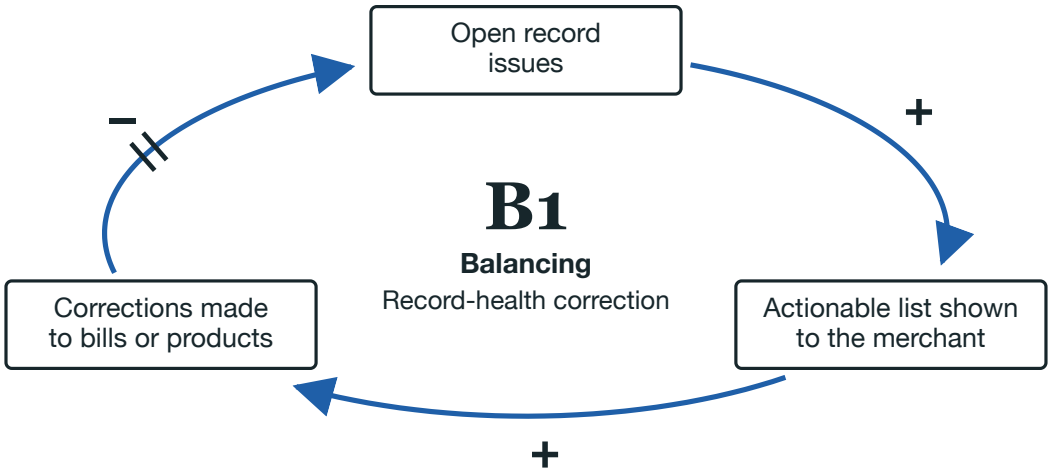


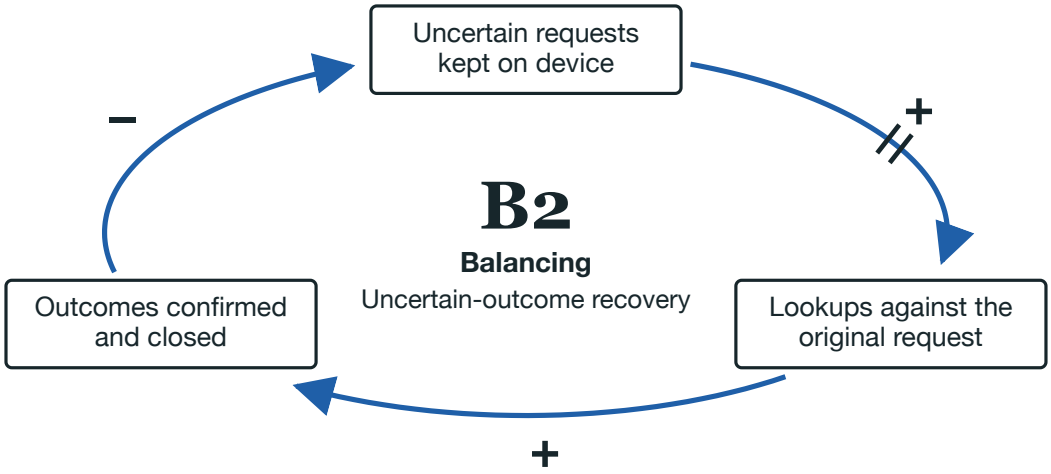
Figure 2. Loops on top of the stock-and-flow diagram. Blue circles mark balancing loops: B1 regulates Open record issues, B2 regulates Uncertain requests, B3 regulates Invoice numbers on the device.

Notation in the loop diagrams that follow: a **+** means the two ends move the same way, a **–** means they move opposite ways. Two short bars across an arrow mean a delay. A loop with an odd number of minus signs is balancing; with none (or an even number) it is reinforcing.



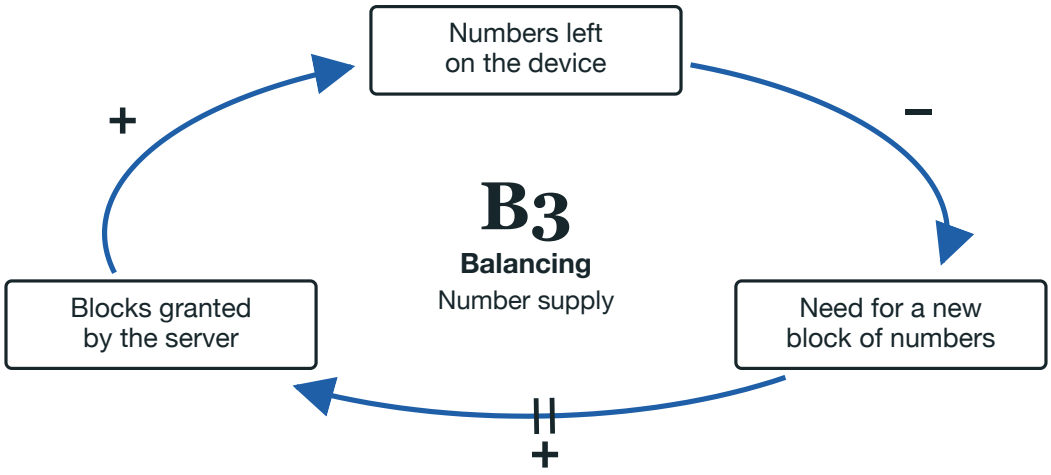
The more issues exist, the more the checks page has to show, the more the merchant corrects, the fewer issues remain. The delay (bars) is real: a fix only disappears from the list when the checks run again. The design rule that a failed or truncated check must never show all clear protects this loop's information link.

DOCS

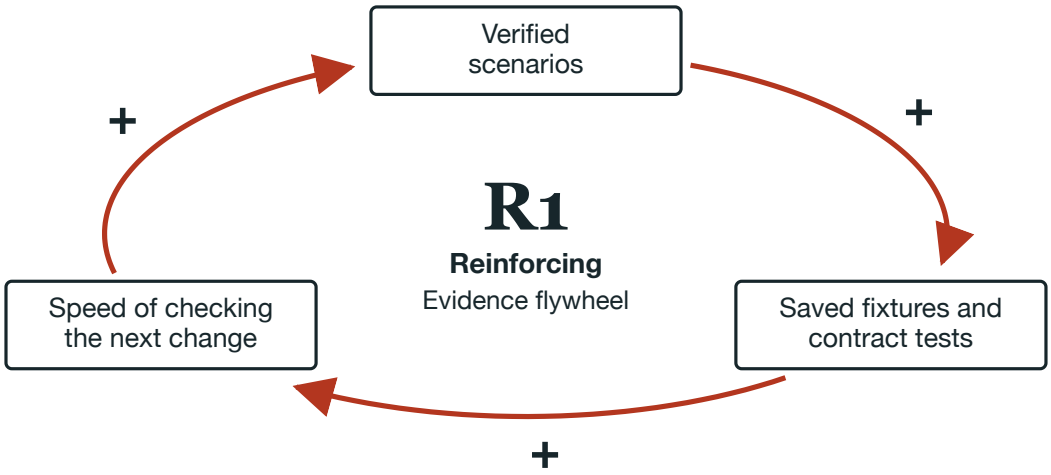


A lost reply creates uncertainty; the journal keeps the original request identity so a lookup can confirm what the server stored. The delay is waiting for the app to restart or the network to return. The plan forbids automatic replay or deletion of ambiguous requests, so this loop is slow but never creates duplicates.

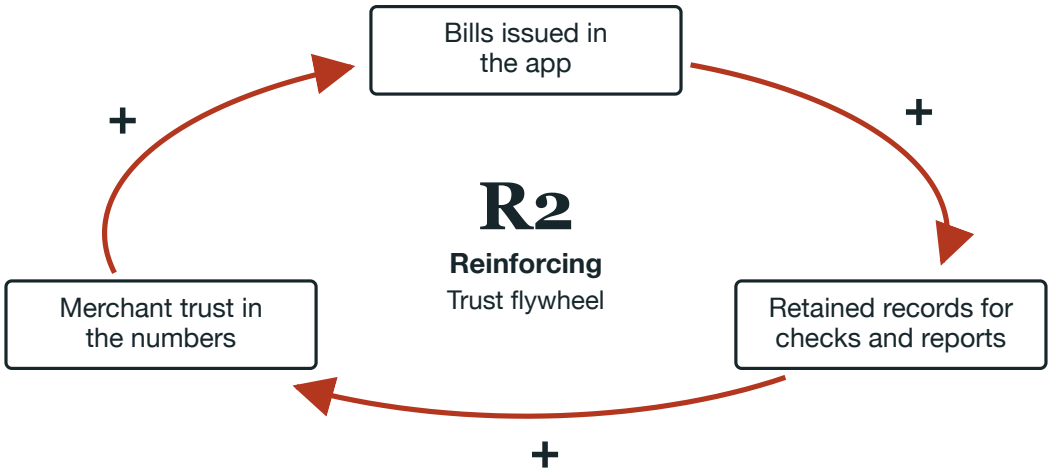
CODE DOCS



As numbers run down, the need for a new block grows, and a granted block refills the stock. The delay is offline time before the next sync. The block structure and the 1–100 index come from the code; that the app requests blocks as a stock runs low is my reading of the design. CODE INFERENCE



Each verified scenario leaves fixtures and tests behind, which make the next change cheaper to check. The repository shows this: the 32 vendored contracts and a drift-check script, plus hundreds of dated evidence folders on the mobile side. It also runs backward: unverified areas get no fixtures and stay slow to verify. DOCS



More bills in the app produce a fuller record for checks and exports, which makes the numbers more trustworthy, which brings more billing into the app. Reinforcing loops cut both ways: one visibly wrong bill can reverse this loop faster than many right bills build it. INFERENCE

Behaviour-over-time graphs

Each graph shows the shape a stock should produce given the loops above, before asking why. **These are illustrations of shape, not measurements.** Samaan Bol has no production data in the repository, so the vertical scale is unlabelled except where the code fixes a number (the block of 100).

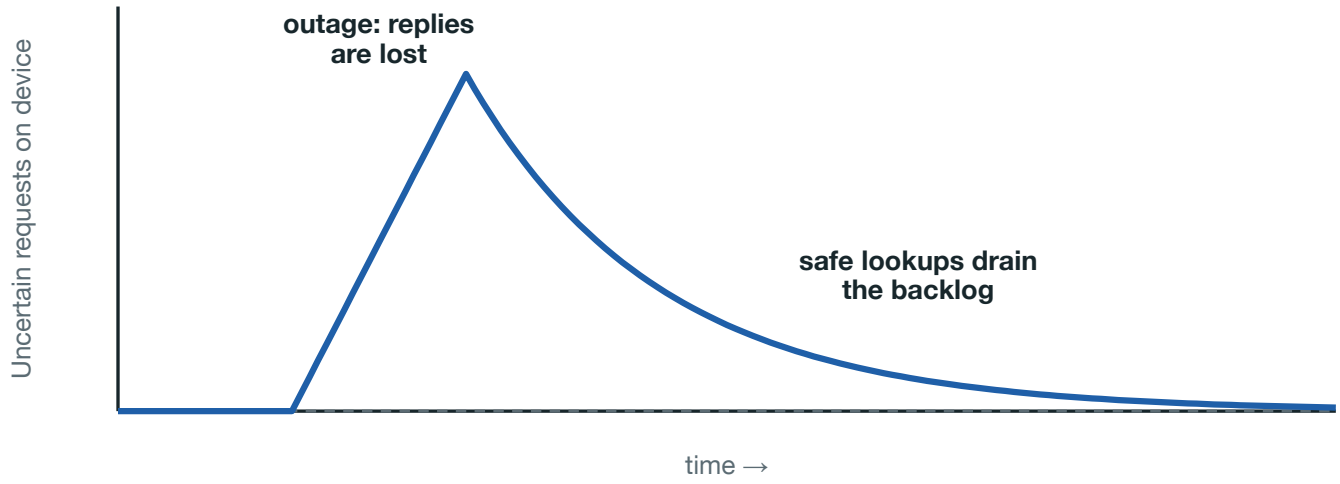


Figure 3. Uncertain requests after a network outage. The stock rises sharply while replies are lost, then falls smoothly toward zero as lookups clear it. This is goal-seeking behaviour from loop B2. The decay slows as it approaches zero because each cleared request is a smaller share of the remaining work.

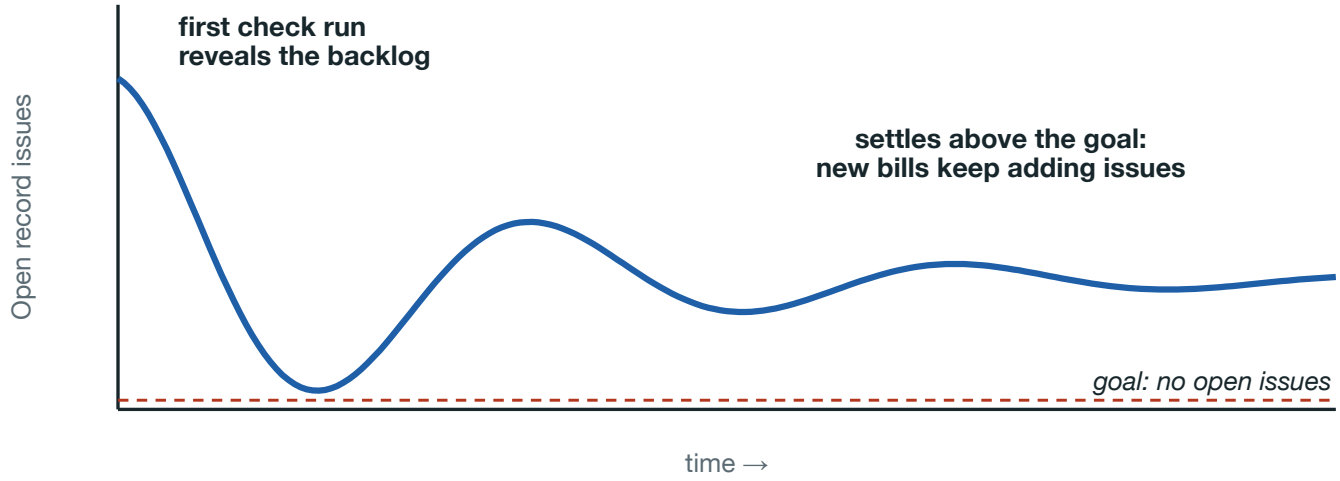


Figure 4. Open record issues. The first check run exposes a backlog. Corrections overshoot and the count swings before settling, a typical result of a balancing loop with a delay (B1). The stock does not reach zero, because every new bill can create new issues. Reaching the dashed goal would need either fewer issue sources or faster correction.

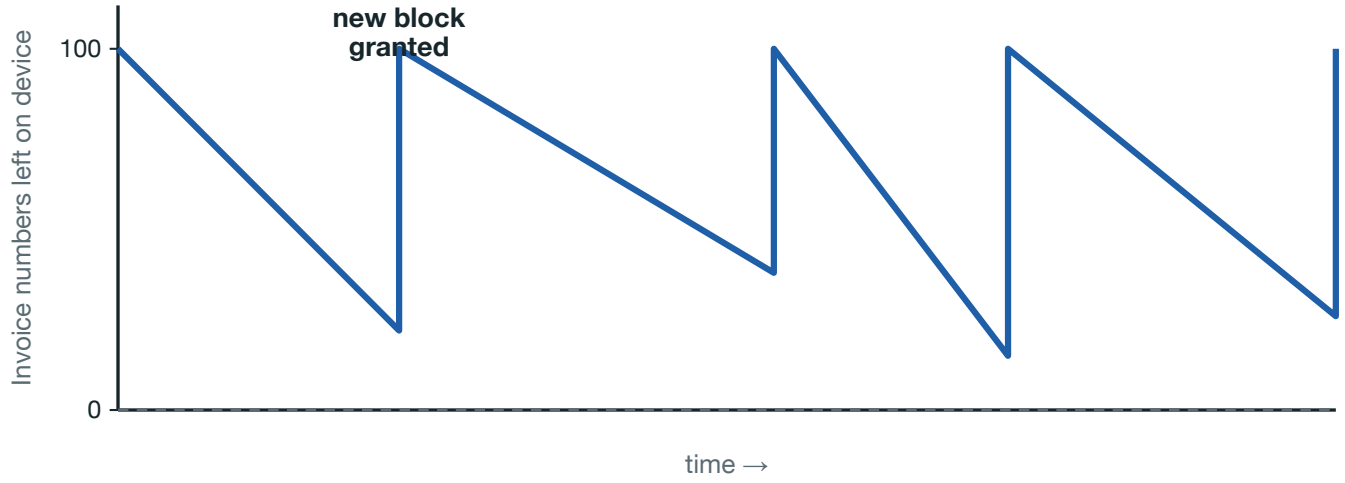


Figure 5. Invoice numbers left on one device. The stock falls with each bill and jumps back when the server grants a new block of up to 100. This sawtooth is the signature of loop B3. If a block runs out offline, billing on that device stops until sync, which is why block size acts as a buffer.

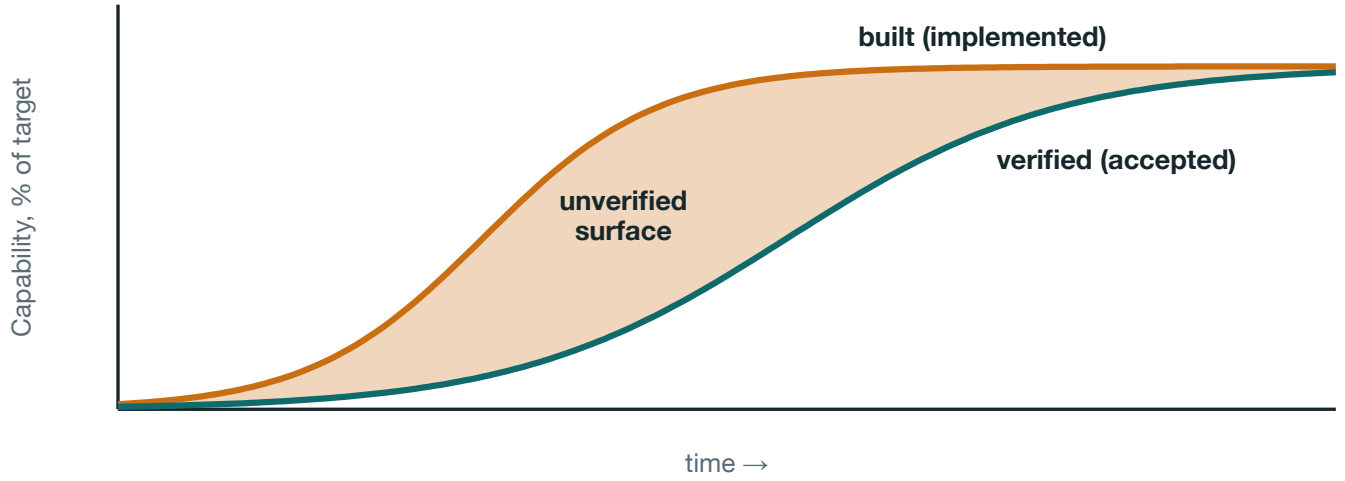


Figure 6. Built versus verified capability. Features were implemented faster than they could be accepted, so the shaded gap grows before it narrows. Today the web parity work is in that gap: implemented locally, with authenticated acceptance still open. The curves are a sketch of the pattern, not a schedule.

TOOL 4

System archetypes and their way out

Archetypes are recurring structures that produce the same problem in very different places. The cards match each one to Samaan Bol, say how strong the match looks, and give the standard way out. Where the repository gives no sign of a trap, I say so.

Tragedy of the commons

MATCH: MANAGED

THE PATTERN Many users draw on a shared resource, and nobody feels the cost of overuse.

IN SAMAAN BOL One invoice-number series per shop is shared by several phones, browsers and tabs. Without control, two devices could take the same number, or someone could reuse an unconfirmed one.

EVIDENCE CODE DOCS Numbers come in blocks tied to a device epoch, can be revoked or expire, and the plan forbids any option to reuse or reclaim unconfirmed numbers.

Way out. Give each user a private share and make the cost visible. Already done with per-device blocks. Keep the rule that no screen ever offers to reclaim numbers.

Policy resistance

MATCH: WATCH

THE PATTERN A rule meets people with different goals, who push back until the rule's effect disappears.

IN SAMAAN BOL Interstate sales and shops that require e-invoicing are blocked. Merchants still have to sell, so they may bill those sales somewhere else. The block works technically but moves real GST activity outside the retained record.

EVIDENCE INFERENCE The blocks are in code and in ten-language error strings. What merchants do after being blocked is not measured.

Way out. Align with the merchant's goal. State the supported boundary at setup, give a clear route for blocked sales, and ask real shops what they do today.

Drift to low performance

MATCH: WATCH

THE PATTERN Standards follow past results, so every shortfall lowers the next target.

IN SAMAAN BOL The Core Release scope document moves purchases, input credit, rounding, providers and the practitioner-approval gate into 'removed from this goal', while many acceptance rows stay pending. Narrowing scope can be a sound decision. It becomes drift if each missed target is answered by lowering the target.

EVIDENCE DOCS See the P0 to P9 table in `GST-core-scope-classification.md`.

Way out. Hold absolute standards. Fix the acceptance matrix in advance (the web plan has one), judge against it and not against what passed last time, and date-stamp every descoped item so it gets revisited.

Escalation

MATCH: NOT OBSERVED

THE PATTERN Two parties each answer the other's move with a bigger one.

IN SAMAAN BOL Nothing in the repository shows a feature race. It would appear if Samaan Bol started matching rivals on e-invoicing, e-way bills or return filing just to keep pace.

EVIDENCE INFERENCE No evidence found.

Way out. De-escalate unilaterally. Decide the positioning (trustworthy counter billing) and measure against that, not against a rival's feature list.

Success to the successful

MATCH: STRONG

THE PATTERN The winner gets the resources that make it win again, starving the rest.

IN SAMAAN BOL Same-state mobile sales has hundreds of dated evidence folders under `docs/gst-core-*`. Purchases, input credit, web roles and cross-client readback have almost none, and manager and helper tests are blocked for lack of test sessions. The best-proven path attracts more proof.

EVIDENCE DOCS Blocked rows C1 and C6 in the acceptance register; web plan lists acceptance as outstanding.

Way out. Ration capacity. Reserve fixed acceptance effort for the starved flows (staging login, manager and helper sessions), or hide the unproven screens until they earn a place.

Shifting the burden

MATCH: MODERATE

THE PATTERN A quick fix relieves the symptom and weakens the real cure.

IN SAMAAN BOL Two cases. First, the app accepts any correctly formatted HSN code and any rate between 0 and 100, so the merchant types them and carries the risk of getting them wrong. There is no reviewed rate source. Second, exports are 'preparation' CSV files, and special invoices fail closed to JSON export, which relieves pressure to build filing and special printing.

EVIDENCE CODE DOCS `validateProductTax` checks format only; web plan states special payloads fall back to JSON export.

Way out. Strengthen the fundamental fix: a reviewed HSN and rate lookup, or an accountant-review step. Label the symptomatic fixes as temporary in the product so no one depends on them.

Rule beating

MATCH: MODERATE

THE PATTERN People obey the letter of a rule and defeat its purpose.

IN SAMAAN BOL A bill total of ₹50,000 or more needs the recipient's name and address. Splitting one sale into two bills under that line beats the rule. Likewise any 4, 6 or 8 digit code passes the code check whether or not it is the right one.

EVIDENCE CODE Threshold at `gst.ts` line 261; code pattern at `validateProductTax`.

Way out. Design rules around purpose. Add a record check that flags clusters of bills just under the threshold, validate codes against a reference list, and explain why each rule exists.

Seeking the wrong goal

MATCH: STRONG

THE PATTERN The measured proxy replaces the real goal, and the system optimises the proxy.

IN SAMAAN BOL Build, type checks and 32 matching contracts show that web equals mobile. The web plan itself says these do not prove cross-client behaviour. The real goal is that a merchant's records are right and ready to file, and practitioner approval was dropped as a release gate.

EVIDENCE DOCS Web plan 'Verified locally' paragraph; scope document row 'practitioner-approved scenarios: Removed'.

Way out. State the goal as correct tax outcomes. Add a measure for it: a sample of real bills reviewed by a GST practitioner, plus mismatches reported by merchants or accountants, and treat it as a release gate or a regular audit.

TOOL 5

Leverage points checklist

Meadows ranks twelve places to intervene. The list runs from weakest (number 12, adjusting parameters) to strongest (number 1, rising above any single paradigm). The table puts each level next to what Samaan Bol already has and what a next move could be. Use it as a checklist: before changing a number, ask whether a higher level would do the job.

RANK	LEVERAGE POINT	IN SAMAAN BOL TODAY CODE DOCS	READING AND NEXT MOVE INFERENCE
12	**Numbers, parameters, standards**	Rate per product (0 to 100, two decimals), ₹50,000 recipient threshold, up to 100 numbers per block.	Weakest lever. Tweaking these changes little. Do not spend effort here first.
11	**Size of buffers**	Numbers held on the device between syncs; browser journal of uncertain requests; unconfirmed numbers that are never reclaimed.	Block size trades offline resilience against numbers at risk if a device is lost. Tune it with real offline data.
10	**Structure of stocks and flows**	One backend and one set of shop records for web and mobile; no second GST database. Pure contracts vendored into web instead of imported from a sibling folder.	Already well chosen. Protect it: any parallel store would split the truth.
9	**Length of delays**	Checks show a fix only on the next run; period close happens after the fact; web parity trailed mobile; acceptance waits on a staging login.	Cheapest high-impact move: stand up a staging auth environment so acceptance stops waiting.
8	**Strength of balancing loops**	Record checks (B1), verified-outcome recovery (B2), number reservation (B3). A failed check may never show all clear.	Strong. Confirm whether checks run automatically or only when the merchant opens the page.
7	**Gain of reinforcing loops**	Evidence flywheel (R1) and trust flywheel (R2).	Feed R1. Guard R2: one visibly wrong bill damages trust faster than many right ones build it.
6	**Structure of information flows**	Live capability read from the API; failed reads kept distinct from empty ones; each invoice stores the profile version used; drift check for vendored contracts.	The best-used lever in the project. Gap: unsupported cases (interstate, e-invoice) reach the merchant only as an error message.
5	**Rules of the system**	Fail closed on unsupported supply; issued payloads never rewritten; owner-only declarations; no automatic replay of ambiguous requests.	Strong. Gap: purchases, input credit and rounding tools are live on web though descoped on mobile. A rule should gate or label them.
4	**Power to change structure**	Server capabilities switch workflows on and off; a feature-off runbook; versioned contracts.	Good. New tax rules (cess, interstate) need a change to the shared core, by design.
3	**Goals**	Core Release goal: exact-paise same-state counter billing, 'prepare, not file'.	Biggest open decision. Make the goal measurable as correct outcomes (see the wrong-goal trap).

2	**Mindset or paradigm**	The merchant confirms the data; the app validates and fails closed. Retain, never erase. Prepare for external filing.	This is the product's character. Its weak point is the assumption that the merchant knows the right code.
1	**Power to transcend paradigms**	The tax regime is a versioned layer (`GST_VERSION`) over a general record of trade.	Long-term only. It matters if the product ever serves another tax regime. No action now.

> **Where to act first, strongest lever first.**
>
> 1. **Goal (3).** Define success as correct tax outcomes and add a practitioner-reviewed sample of real bills as a gate or recurring audit.
> 2. **Information (6).** Tell merchants the supported boundary at setup and at the moment a bill is blocked, with a next step.
> 3. **Rules (5).** Gate or label the web purchase, input-credit and rounding screens until they are accepted.
> 4. **Delays (9).** Stand up a staging login and close the manager and helper role tests.
> 5. **Traps.** Add a reviewed HSN and rate lookup (shifting the burden) and a just-under-threshold flag in record checks (rule beating).

TOOL 6

Guidelines for living in a world of systems

Meadows closes with habits rather than a diagram. This is a working stance. Each guideline below is paired with one concrete thing it asks of the Samaan Bol team, drawn from the project as it stands.

1 **Get the beat of the system**
Watch before changing. Read a few weeks of real checks, recovery and export history from live shops: how often do numbers run out, how often do replies get lost, which issues repeat? Today the evidence comes from staged fixtures.

2 **Expose your mental models to the open air**
The scope documents, the parity plan and the diagrams in this PDF are models. Show them to a GST practitioner and a shopkeeper and let them correct what is wrong.

3 **Honor, protect and distribute information**
Keep failed reads distinct from empty results (already a rule). Show who changed a profile and which version a bill used. Surface unsupported boundaries before the merchant hits them.

4 **Use language with care**
Keep 'prepared for filing' apart from 'filed', 'unconfirmed' apart from 'unused', and 'deferred setup' apart from 'unregistered'. The plan already does this. Native-speaker review of the ten languages is still open.

5 **Pay attention to what is important, not just what is quantifiable**
Thirty-two matching contracts is easy to count. 'A shopkeeper files with confidence' is not. Add a qualitative check next to the countable ones.

6 **Make feedback policies for feedback systems**
Do not hard-code a fix per case. Strengthen the loops: the record checks, the verified replay, the drift check. The rule that a failed run never shows all clear is a good example.

7 **Go for the good of the whole**
A bill serves the shop, its customer, the accountant and the tax record. Do not trade tax correctness for checkout speed.

8 **Listen to the wisdom of the system**
A block such as `tax_profile_missing` is the system saying the data is not ready. Resist bypassing it. Signs of rule beating show where rules clash with real shops.

9 **Locate responsibility in the system**
Merchants type HSN codes and rates because the product offers no reviewed source. Part of that responsibility sits in the design, not only in the user.

10 **Stay humble, stay a learner**
The web plan says the work 'must not be labelled complete or mobile-equivalent based on the build alone'. Keep that candour. Staging acceptance and language review are the learning steps still ahead.

11 **Celebrate complexity**
Indian GST has composition schemes, exempt, nil and non-GST goods, and union territories with their own tax head. Keep the boundaries explicit instead of promising universal support.

12 **Expand time horizons**
Invoices, credit notes, period closes and exports have to stay readable across financial years. Judge a change by what an audit will see later, not only by today's checkout.

13 **Defy the disciplines**
This feature is tax law, shopkeeper usability, offline-first engineering and ten languages at once. Keep practitioners, shopkeepers and engineers in the same review.

14 **Expand the boundary of caring**
Include the customer who receives the bill, the accountant who files, the supplier whose GSTIN is stored, and the helper or manager with limited access.

15 **Don't erode the goal of goodness**
Do not pass a gate by narrowing it, and do not weaken login to make a local preview work. The parity plan explicitly refuses the second. Hold both lines.

Limits of this document: it is based on source and documents, not live data or interviews. I did not run the mobile test suite. Loops and archetype matches tagged Inference are hypotheses to test with real merchants, not findings.